

Curso de Programação Básica em Python

Público-alvo: iniciantes absolutos

Carga horária total: 8 horas

Formato: 8 aulas de 1 hora

Objetivo geral: desenvolver, de forma progressiva, a capacidade de compreender e produzir programas simples em Python, dominando os fundamentos da linguagem e os pilares iniciais da programação.

Progressão pedagógica do curso

A sequência proposta segue uma evolução natural:

1. contato inicial com Python,
2. manipulação de dados,
3. entrada e processamento,
4. decisões lógicas,
5. repetição de tarefas,
6. modularização com funções,
7. armazenamento com estruturas de dados,
8. integração e boas práticas.

Resultado esperado ao final do curso

Ao concluir as 8 aulas, o estudante deverá ser capaz de:

- escrever programas simples em Python;
- usar variáveis, operadores e tipos de dados corretamente;
- construir estruturas condicionais e laços de repetição;
- criar funções para organizar o código;
- manipular listas, tuplas e dicionários;
- aplicar noções básicas de boas práticas de programação.

Table of Contents

Aula 1 — Introdução à programação e primeiros passos com Python – 06/05/2026	3
Aula 2 — Variáveis e tipos de dados básicos – 13/05/2026	4
Aula 3 — Operadores e entrada de dados – 20/05/2026	5
Aula 4 — Estruturas condicionais – 27/05/2026	7
Aula 5 — Estruturas de repetição – 03/05/2026	8
Aula 6 — Funções – 10/05/2026	9
Aula 7 — Estruturas de dados básicas: listas, tuplas e dicionários – 17/05/2026	10
Aula 8 — Integração dos conceitos e boas práticas introdutórias – 24/05/2026	12

Aula 1 — Introdução à programação e primeiros passos com Python - 06/05/2026

Objetivos de aprendizagem

Ao final da aula, o(a) estudante deverá ser capaz de:

- **Explicar** o conceito de programação e sua aplicação na resolução de problemas.
- **Reconhecer** Python como linguagem de programação de alto nível, de uso amplo e sintaxe acessível.
- **Identificar** os principais elementos do ambiente de execução: interpretador, editor de código e terminal.
- **Descrever** a estrutura mínima de um programa em Python.
- **Utilizar** a função `print()` para produzir saída de dados na tela.
- **Inserir** comentários no código com a finalidade de documentação básica e organização textual.
- **Executar** um script simples contendo comandos elementares da linguagem.

Conteúdo teórico essencial

- Conceito de programação como processo de instrução lógica para resolução de problemas.
- Visão geral da linguagem Python: características, finalidade e aplicabilidade.
- Ambiente de desenvolvimento: editor de código, interpretador e terminal.
- Estrutura mínima de um programa em Python.
- Noções iniciais de sintaxe e execução de scripts.
- Função `print()` como mecanismo de saída de dados.
- Comentários em Python: uso de `#` para anotação e legibilidade do código.

Atividade prática / exercício

- Configurar e organizar a pasta inicial do curso ou do mini projeto.
- Criar um primeiro script Python com mensagens de apresentação.
- Utilizar `print()` para exibir:
 - nome do estudante;
 - nome do curso;
 - uma frase de apresentação personalizada.
- Inserir comentários no código para identificar cada bloco funcional.
- Executar o script no ambiente de desenvolvimento e observar a saída gerada.

Objetivos de aprendizagem

Ao final da aula, o(a) estudante deverá ser capaz de:

- **Definir** variáveis como estruturas de armazenamento de valores em um programa.
- **Atribuir** valores a variáveis utilizando o operador `=`.
- **Identificar** os tipos de dados básicos em Python.
- **Diferenciar** valores do tipo `str`, `int`, `float` e `bool`.
- **Utilizar** variáveis para armazenar e recuperar informações simples.
- **Aplicar** a função `type()` para verificar o tipo de dado armazenado em uma variável.
- **Escrever** nomes de variáveis seguindo critérios básicos de clareza e legibilidade.

Conteúdo teórico essencial

- Conceito de variável como identificador associado a um valor armazenado na memória.
- Operador de atribuição `=`.
- Tipos de dados básicos em Python:
 - `str` — cadeia de caracteres (texto);
 - `int` — número inteiro;
 - `float` — número de ponto flutuante;
 - `bool` — valor lógico (`True` / `False`).
- Função `type()` para inspeção de tipo.
- Boas práticas iniciais de nomenclatura:
 - uso de nomes descritivos;
 - ausência de espaços;
 - preferência por letras minúsculas e sublinhado (`snake_case`).

Atividade prática / exercício

- Criar variáveis para armazenar:
 - nome;
 - idade;
 - altura;
 - status de matrícula.
- Exibir os valores dessas variáveis com `print()`.
- Aplicar `type()` em cada variável para identificar seu tipo de dado.
- Registrar, em comentários no código, a função de cada variável.

Objetivos de aprendizagem

Ao final da aula, o(a) estudante deverá ser capaz de:

- **Aplicar** operadores aritméticos em expressões numéricas simples.
- **Comparar** valores utilizando operadores relacionais.
- **Interpretar** expressões booleanas construídas com operadores lógicos.
- **Coletar** dados digitados pelo usuário por meio da função `input()`.
- **Converter** entradas textuais para tipos numéricos com `int()` e `float()`.
- **Construir** interações elementares com o usuário em um programa Python.
- **Distinguir** entrada textual de valor numérico para evitar inconsistências em cálculos.

Conteúdo teórico essencial

- Operadores aritméticos:
 - + adição
 - - subtração
 - * multiplicação
 - / divisão
 - // divisão inteira
 - % módulo
 - ** exponenciação
- Operadores relacionais:
 - ==, !=, >, <, >=, <=
- Operadores lógicos:
 - and, or, not
- Função `input()` como mecanismo de entrada de dados.
- Conversão de tipos com `int()`, `float()` e `str()`.
- Diferença entre valores recebidos como texto e valores numéricos utilizados em operações matemáticas.
- Noções iniciais de validação implícita de dados por meio da conversão.

Atividade prática / exercício

- Criar um programa que solicite ao usuário:
 - nome e idade.
- Converter a idade recebida para `int`.
- Exibir uma mensagem personalizada com o nome e a idade informada.
- Resolver pequenas expressões com operadores matemáticos, explorando pelo menos:
 - uma operação de soma;
 - uma operação com divisão inteira ou módulo;
 - uma comparação lógica simples.

- Registrar comentários no código para identificar a função de cada bloco.

Objetivos de aprendizagem

Ao final da aula, o(a) estudante deverá ser capaz de:

- **Compreender** o papel das estruturas condicionais na tomada de decisão computacional.
- **Aplicar** as estruturas `if`, `elif` e `else` em situações simples.
- **Construir** expressões condicionais utilizando operadores relacionais e lógicos.
- **Organizar** blocos de código com indentação adequada.
- **Analisar** diferentes cenários de entrada para validar o comportamento do programa.
- **Classificar** valores com base em critérios previamente definidos.

Conteúdo teórico essencial

- Conceito de estrutura condicional como mecanismo de controle de fluxo.
- Blocos de código e indentação em Python.
- Estrutura `if` para execução condicional simples.
- Estrutura `if/else` para seleção entre duas possibilidades.
- Estrutura `if/elif/else` para múltiplas condições mutuamente exclusivas.
- Uso de operadores relacionais e lógicos na composição de testes condicionais.
- Noção de fluxo de execução com base em decisão programada.

Atividade prática / exercício

- Criar um programa que solicite a idade de uma pessoa e verifique se ela é maior de idade.
- Desenvolver um sistema simples de classificação por nota, com as seguintes faixas:
 - aprovado
 - recuperação
 - reprovado
- Testar o programa com diferentes valores de entrada para verificar a consistência das saídas.
- Registrar comentários no código para explicitar a lógica aplicada em cada bloco condicional.

Aula 5 — Estruturas de repetição - 03/05/2026

Objetivos de aprendizagem

- Entender por que e quando usar repetições.
- Utilizar `while` e `for`.
- Repetir ações de forma controlada.
- Aplicar laços em problemas simples.

Conteúdo teórico essencial

- Conceito de repetição em programação.
- Estrutura `while`.
- Estrutura `for`.
- Função `range()`.
- Controle de repetição com condições.
- Noção inicial de contagem e iteração.

Atividade prática/exercício

- Criar um contador regressivo com `while`.
- Exibir uma sequência numérica com `for` e `range()`.
- Fazer um programa que repete uma mensagem três vezes.
- Montar uma tabuada simples de um número informado.

Objetivos de aprendizagem

Ao final da aula, o(a) estudante deverá ser capaz de:

- **Explicar** a função das funções na organização de programas.
- **Definir** funções utilizando a estrutura `def`.
- **Distinguir** parâmetros de argumentos na chamada de funções.
- **Retornar** valores com a palavra-chave `return`.
- **Diferenciar** funções com retorno e funções sem retorno.
- **Reutilizar** blocos de código por meio de chamadas repetidas de funções.
- **Aplicar** funções simples em tarefas computacionais elementares.

Conteúdo teórico essencial

- Conceito de função como subrotina ou bloco de código nomeado.
- Vantagens da utilização de funções: reaproveitamento, organização, manutenção e legibilidade.
- Sintaxe de definição de funções com `def`.
- Parâmetros formais e argumentos na chamada da função.
- Palavra-chave `return` como mecanismo de devolução de valores.
- Diferença entre funções com retorno e funções que apenas executam ações.
- Noção de encapsulamento lógico de tarefas em blocos independentes.

Atividade prática / exercício

- Criar funções para:
 - somar dois números;
 - verificar se um número é par;
 - exibir uma saudação personalizada.
- Chamar as funções com diferentes valores de entrada.
- Observar e registrar a diferença entre:
 - função que retorna um valor;
 - função que apenas produz uma saída na tela.
- Testar as funções em cenários variados para verificar o comportamento esperado.

Objetivos de aprendizagem

Ao final da aula, o(a) estudante deverá ser capaz de:

- **Compreender** o papel das estruturas de dados na organização de informações.
- **Criar** listas, tuplas e dicionários em Python.
- **Acessar** elementos utilizando índices e chaves.
- **Diferenciar** estruturas mutáveis e imutáveis.
- **Manipular** listas por meio de inserção e remoção de elementos.
- **Atualizar** e **consultar** dados em dicionários.
- **Percorrer** coleções utilizando estruturas de repetição.
- **Selecionar** a estrutura de dados mais adequada para diferentes contextos.

Conteúdo teórico essencial

- Conceito de estrutura de dados como mecanismo de organização e armazenamento de coleções de valores.
- **Listas (list):**
 - criação com colchetes [];
 - acesso por índice;
 - mutabilidade;
 - métodos básicos: adição (append()), remoção (remove(), pop()).
- **Tuplas (tuple):**
 - criação com parênteses ();
 - acesso por índice;
 - imutabilidade;
 - uso em contextos de dados fixos.
- **Dicionários (dict):**
 - estrutura de pares chave–valor;
 - criação com chaves { };
 - acesso por chave;
 - inserção e atualização de valores;
 - métodos básicos (get(), atribuição direta).
- Comparação entre listas, tuplas e dicionários:
 - organização dos dados;
 - possibilidade de modificação;
 - forma de acesso aos elementos.

Atividade prática / exercício

- Criar uma lista contendo nomes de alunos.
- Adicionar e remover elementos da lista.
- Criar uma tupla com dados fixos (dia, mês e ano).
- Acessar elementos da tupla por índice.

- Criar um dicionário com informações (nome, idade e curso).
- Atualizar um dos valores do dicionário.
- Percorrer a lista utilizando um laço `for` para exibir seus elementos.
- Inserir comentários no código explicando a finalidade de cada estrutura.

Objetivos de aprendizagem

Ao final da aula, o(a) estudante deverá ser capaz de:

- **Integrar** variáveis, operadores, estruturas condicionais, estruturas de repetição, funções e estruturas de dados em um único programa.
- **Organizar** o código em blocos lógicos coerentes.
- **Aplicar** boas práticas básicas de escrita de código em Python.
- **Nomear** variáveis e funções de forma clara e significativa.
- **Documentar** o código com comentários objetivos e funcionais.
- **Estruturar** a separação entre entrada, processamento e saída de dados.
- **Desenvolver** uma solução simples de forma autônoma, aplicando os conceitos aprendidos.

Conteúdo teórico essencial

- Revisão sistematizada dos principais conceitos do curso:
 - variáveis e tipos de dados;
 - operadores;
 - entrada de dados;
 - estruturas condicionais;
 - estruturas de repetição;
 - funções;
 - estruturas de dados.
- Organização do código:
 - decomposição em blocos lógicos;
 - separação de responsabilidades.
- Boas práticas introdutórias:
 - nomenclatura significativa (`snake_case`);
 - padronização e legibilidade;
 - uso adequado de comentários (quando agregam valor explicativo);
 - indentação consistente.
- Estrutura básica de programas:
 - entrada de dados;
 - processamento;
 - saída de resultados.
- Noções iniciais de manutenção e legibilidade de código.

Atividade prática / exercício

- Desenvolver, em sala, um mini projeto integrador, com opções como:
 - cadastro simples de alunos;
 - controle básico de notas;
 - lista de tarefas.

- O programa deve obrigatoriamente contemplar:
 - uso de variáveis para armazenamento de dados;
 - aplicação de estruturas condicionais;
 - utilização de pelo menos uma estrutura de repetição;
 - definição e uso de funções;
 - uso de lista ou dicionário para organização dos dados.
 - Realizar testes com diferentes entradas para validar o funcionamento do programa.
 - Participar de uma revisão orientada do código, com foco em:
 - clareza na nomeação;
 - organização estrutural;
 - melhoria da legibilidade;
 - identificação de possíveis redundâncias ou inconsistências.